

Structures de Données et Algorithmes Avancés

Lab S04 — Arbres et tas

Implémenter un BST, observer la hauteur

Master 1 GLSI — 2025–2026

Dr. El Hadji Bassirou TOURÉ
École Supérieure Polytechnique
Département Génie Informatique
Université Cheikh Anta Diop de Dakar

Prérequis et contexte

Prérequis : CM S04 (BST, AVL, tas binaire, percolation). Lab S01 complété (Docker, Gradle, tests JUnit).

Objectif : Implémenter un `SimpleBST<K,V>` générique et *observer concrètement* ce qu'on a vu en CM : le coût $\mathcal{O}(h)$ des opérations et la différence spectaculaire entre insertions triées (hauteur $n - 1$) et insertions aléatoires (hauteur $\approx 2,99 \ln n$). Puis faire quelques exercices théoriques (BST, AVL, tas) pour consolider.

Organisation : Ce lab est en **deux parties**. Partie I (pratique) : on code et on observe, pas à pas. Partie II (théorique) : des traces papier à dérouler en lien avec ce qu'on a codé et avec le CM.

Livrable : un dépôt Git avec le code + un rapport individuel (1–2 pages) décrivant ce que vous avez fait et appris.

À la fin de ce lab — vous saurez

1. Implémenter `SimpleBST<K,V>` avec `put`, `get`, `contains`, `size`, `height`
2. Écrire des tests JUnit qui *forcent* le cas dégénéré
3. Visualiser empiriquement la différence $h \approx n - 1$ vs $h \approx \log n$
4. Dérouler à la main l'insertion et la suppression dans un BST
5. Identifier le cas d'un déséquilibre AVL (LL, RR, LR, RL)
6. Manipuler la représentation d'un tas en tableau et tracer une percolation

Table des matières

I	Pratique : implémentation et observation	2
1	Environnement de travail	2
1.1	Créer le projet	2
2	Étape 1 — L'interface <code>BinarySearchTree<K,V></code>	2
3	Étape 2 — Implémenter <code>SimpleBST</code>	3

4	Étape 3 — Écrire des tests JUnit	4
5	Étape 4 — Observer : hauteur selon l'ordre d'insertion	6
6	Étape 5 — Commit Git	8
 II Théorie : exercices d'application		9
7	Exercice 1 — Insertion et suppression dans un BST	9
8	Exercice 2 — Hauteur et complexité	11
9	Exercice 3 — Rotations AVL	11
10	Exercice 4 — Arithmétique des indices dans un tas	12
11	Exercice 5 — Percolation dans un min-heap	12
12	Exercice 6 — Pièges conceptuels	13
13	Pour aller plus loin (optionnel)	13
14	Aide-mémoire	13

Première partie

Pratique : implémentation et observation

1 Environnement de travail

Rappel Docker

Ce lab se fait dans l'environnement Docker du cours.

Lancement :

```
# Linux/macOS
docker run -it -v "$(pwd):/workspace" elbachirtoure/dsa-java-m1:v1.1

# Windows (Git Bash)
MSYS_NO_PATHCONV=1 docker run -it -v "$(pwd):/workspace" \
    elbachirtoure/dsa-java-m1:v1.1

cd /workspace
```

Si vous n'avez jamais utilisé cet environnement, revoyez le TP-1.

1.1 Créer le projet

```
cd /workspace
new-project lab-s04
cd lab-s04
```

La commande `new-project` crée un squelette Gradle avec JUnit 5 configuré, un `Main.java`, un `MainTest.java` et un dossier `src/main/java/`.

2 Étape 1 — L'interface `BinarySearchTree<K,V>`*Ce qu'on matérialise du CM*

En S04 on a défini le BST par un contrat (*put*, *get*, *contains*, *size*) plus une méthode *height* pour observer la hauteur. C'est la hauteur qui fait toute la différence entre $\mathcal{O}(\log n)$ et $\mathcal{O}(n)$.

Créez le fichier `src/main/java/BinarySearchTree.java`.

```
1  /**
2   * ADT Arbre Binaire de Recherche (BST) generique.
3   * @param <K> type des cles (doit etre Comparable pour definir l'ordre)
4   * @param <V> type des valeurs associees
5   */
6  public interface BinarySearchTree<K extends Comparable<K>, V> {
7
8      /** Insere la paire (cle, valeur). Si cle existe deja, remplace la
9       *   valeur. */
10     void put(K cle, V valeur);
11
12     /** Retourne la valeur associee a la cle, ou null si absente. */
13     V get(K cle);
14
15     /** Teste l'appartenance de la cle. */
16     boolean contains(K cle);
```

```

16
17     /** Nombre de cles stockees. */
18     int size();
19
20     /** Hauteur : arbre vide = -1, feuille = 0. */
21     int height();
22 }

```

Pourquoi exposer height dans l'interface ?

En production, on ne montre pas `height` aux utilisateurs : c'est un détail d'implémentation. Mais pour ce lab, on en a besoin comme *instrument de mesure* : c'est `height` qui va nous permettre de *voir* si notre BST est équilibré ou dégénéré. Analogie : c'est le thermomètre du BST.

3 Étape 2 — Implémenter SimpleBST

Ce qu'on matérialise du CM

On code un BST non équilibré — exactement la structure du CM dont la hauteur dépend de l'ordre d'insertion. L'implémentation est récursive, directement inspirée de Sedgwick §3.2.

Créez `src/main/java/SimpleBST.java` :

```

1 public class SimpleBST<K extends Comparable<K>, V> implements
   BinarySearchTree<K, V> {
2
3     /** Noeud interne de l'arbre. */
4     private static class Node<K, V> {
5         K cle;
6         V valeur;
7         Node<K, V> left, right;
8         Node(K k, V v) { this.cle = k; this.valeur = v; }
9     }
10
11     private Node<K, V> root = null;
12     private int size = 0;
13
14     @Override
15     public void put(K cle, V valeur) {
16         if (cle == null) throw new IllegalArgumentException("cle null
17             interdite");
18         root = putRec(root, cle, valeur);
19     }
20
21     private Node<K, V> putRec(Node<K, V> n, K cle, V valeur) {
22         if (n == null) {
23             size++;
24             return new Node<>(cle, valeur);
25         }
26         int cmp = cle.compareTo(n.cle);
27         if (cmp < 0) n.left = putRec(n.left, cle, valeur);
28         else if (cmp > 0) n.right = putRec(n.right, cle, valeur);
29         else n.valeur = valeur; // remplacement
30         return n;
31     }
32
33     @Override

```

```

33     public V get(K cle) {
34         if (cle == null) return null;
35         Node<K, V> n = root;
36         while (n != null) {
37             int cmp = cle.compareTo(n.cle);
38             if (cmp < 0) n = n.left;
39             else if (cmp > 0) n = n.right;
40             else return n.valeur;
41         }
42         return null;
43     }
44
45     @Override
46     public boolean contains(K cle) { return get(cle) != null; }
47
48     @Override
49     public int size() { return size; }
50
51     @Override
52     public int height() { return heightRec(root); }
53
54     private int heightRec(Node<K, V> n) {
55         if (n == null) return -1;
56         return 1 + Math.max(heightRec(n.left), heightRec(n.right));
57     }
58 }

```

Piège classique

Dans `putRec`, oublier le `return n` après le `else` (cas remplacement de valeur) est un bug silencieux : Java ne génère aucune erreur, mais l'arbre perd sa racine à chaque remplacement. Le test `testRemplacementValeur` (étape suivante) révélera le bug immédiatement.

Vérification 1 — ça compile

```
gradle --no-daemon build
```

Aucune erreur attendue. Si erreur : lisez attentivement le message, c'est généralement une accolade manquante ou un import oublié.

4 Étape 3 — Écrire des tests JUnit

Ce qu'on matérialise du CM

On teste trois choses : (1) la correction fonctionnelle — ce qu'on met on le retrouve ; (2) le comptage — `size` est cohérent ; (3) le phénomène central du CM — la hauteur dépend de l'ordre d'insertion. Un test spécifique va forcer le cas dégénéré.

Créez `src/test/java/SimpleBSTTest.java` :

```

1  import org.junit.jupiter.api.Test;
2  import static org.junit.jupiter.api.Assertions.*;
3
4  public class SimpleBSTTest {
5
6      @Test

```

```
7 void testPutGet() {
8     SimpleBST<Integer, String> t = new SimpleBST<>();
9     t.put(50, "Dakar");
10    t.put(30, "Thies");
11    t.put(70, "Saint-Louis");
12    assertEquals("Dakar", t.get(50));
13    assertEquals("Thies", t.get(30));
14    assertEquals("Saint-Louis", t.get(70));
15    assertNull(t.get(999));
16 }
17
18 @Test
19 void testContainsEtSize() {
20     SimpleBST<Integer, Integer> t = new SimpleBST<>();
21     assertEquals(0, t.size());
22     t.put(1, 10); t.put(2, 20); t.put(3, 30);
23     assertEquals(3, t.size());
24     assertTrue(t.contains(2));
25     assertFalse(t.contains(4));
26 }
27
28 @Test
29 void testRemplacementValeur() {
30     SimpleBST<Integer, String> t = new SimpleBST<>();
31     t.put(1, "ancien");
32     t.put(1, "nouveau");
33     assertEquals("nouveau", t.get(1));
34     assertEquals(1, t.size(), "size ne doit pas augmenter lors d'un
35         remplacement");
36 }
37
38 @Test
39 void testHauteurEquilibree() {
40     // Ordre d'insertion favorable -> arbre equilibre
41     SimpleBST<Integer, Integer> t = new SimpleBST<>();
42     int[] cles = {50, 30, 70, 20, 40, 60, 80};
43     for (int k : cles) t.put(k, k);
44     // 7 noeuds parfaitement equilibres : hauteur = 2
45     assertEquals(2, t.height());
46 }
47
48 @Test
49 void testForcerCasDegenere() {
50     // Insertions trieées -> arbre degenerate (liste chainee a droite)
51     SimpleBST<Integer, Integer> t = new SimpleBST<>();
52     int n = 10;
53     for (int i = 1; i <= n; i++) t.put(i, i);
54     // Hauteur doit valoir exactement n - 1
55     assertEquals(n - 1, t.height(),
56         "Insertions trieées doivent produire un arbre degenerate");
57 }
```

```
gradle --no-daemon test
```

Vérification 2 — les 5 tests passent

Les 5 tests doivent passer. Le dernier, `testForcerCasDegenere`, est le phénomène central du CM : n insertions triées donnent une hauteur $n - 1$. Tant que ce test ne passe pas, votre `height` est faux.

5 Étape 4 — Observer : hauteur selon l'ordre d'insertion

Ce qu'on matérialise du CM

Le phénomène du déséquilibre. En CM on a annoncé que la hauteur d'un BST peut être soit $\mathcal{O}(\log n)$ soit $\mathcal{O}(n)$ selon l'ordre d'insertion. Ici on va visualiser cet écart sur 3 tailles : $n = 100, 1000, 10000$.

Modifiez `src/main/java/Main.java` :

```

1  import java.util.*;
2
3  public class Main {
4
5      public static void main(String[] args) {
6          int[] tailles = {100, 1000, 10000};
7
8          System.out.println("=== Hauteur d'un BST non equilibre ===");
9          System.out.println();
10         System.out.printf("%-8s | %-22s | %-22s\n",
11                             "n", "h (insertions trieées)", "h (aleatoires,
12                                 seed=42)");
13         System.out.println("
14             -----+-----+-----");
15
16         for (int n : tailles) {
17             int hTrie = hauteurInsertionsTrieées(n);
18             int hAleat = hauteurInsertionsAleatoires(n, 42);
19             System.out.printf("%-8d | %-22d | %-22d\n", n, hTrie, hAleat);
20         }
21
22         System.out.println();
23         System.out.println("Predictions theoriques :");
24         System.out.println("  - Insertions trieées : h = n - 1 (pire cas)");
25         System.out.println("  - Insertions aleatoires : h ~ 2.99 * ln(n)");
26
27         System.out.println();
28         System.out.println("Valeurs predites pour les aleatoires :");
29         for (int n : tailles) {
30             System.out.printf("  n = %-6d -> ~ %.1f\n", n, 2.99 * Math.
31                 log(n));
32         }
33
34         static int hauteurInsertionsTrieées(int n) {
35             SimpleBST<Integer, Integer> t = new SimpleBST<>();
36             for (int i = 1; i <= n; i++) t.put(i, i);
37             return t.height();
38         }
39     }
40 }

```

```

37     static int hauteurInsertionsAleatoires(int n, long seed) {
38         List<Integer> cles = new ArrayList<>();
39         for (int i = 1; i <= n; i++) cles.add(i);
40         Collections.shuffle(cles, new Random(seed));
41         SimpleBST<Integer, Integer> t = new SimpleBST<>();
42         for (int k : cles) t.put(k, k);
43         return t.height();
44     }
45 }

```

Exécution :

```
gradle --no-daemon run
```

Vous devriez obtenir un affichage ressemblant à ceci :

```

=== Hauteur d'un BST non equilibre ===

n          | h (insertions trieées) | h (aleatoires, seed=42)
-----+-----+-----
100        | 99                     | 14
1000       | 999                    | 22
10000      | 9999                   | 32

Predictions theoriques :
- Insertions trieées : h = n - 1 (pire cas)
- Insertions aleatoires : h ~ 2.99 * ln(n)

Valeurs predites pour les aleatoires :
n = 100      -> ~ 13.8
n = 1000     -> ~ 20.7
n = 10000    -> ~ 27.5

```

Vérification 3 — observations à noter

Recopiez les trois lignes de votre tableau :

n	h (insertions triées)	h (insertions aléatoires)
100	_____	_____
1 000	_____	_____
10 000	_____	_____

Questions :

- Pour les insertions triées, vérifiez-vous $h = n - 1$ exactement ?

- Pour $n = 10\,000$ aléatoire, h est-il plus proche de 9999 ou de 27,5 ?

- Écrivez en une phrase ce que ce programme matérialise du CM :

Ce que vous venez de voir en direct

Pour $n = 10\,000$, vos opérations `get` coûtent *9 999 comparaisons* dans le cas trié contre *32* dans le cas aléatoire — un écart de $300\times$. Le pire cas n'est pas théorique : il suffit que les clés arrivent triées (par date, par ID croissant) pour que votre BST dégénère. C'est précisément ce que résout l'AVL (vu en CM), et c'est pourquoi Java utilise des red-black trees dans `TreeMap`.

6 Étape 5 — Commit Git

```
git init
git add .
git commit -m "Lab S04 : SimpleBST avec observation de la hauteur"
```

Deuxième partie

Théorie : exercices d'application

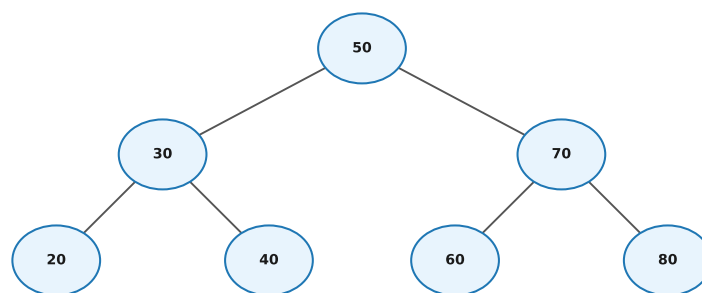
Consigne

Les exercices suivants ne sont pas notés. Ils sont à faire individuellement sur papier, puis discutés en plénière à la fin de la séance. L'objectif n'est pas le résultat mais la démarche — montrer que vous savez dérouler un algorithme à la main.

7 Exercice 1 — Insertion et suppression dans un BST

On considère le BST suivant :

BST initial — à utiliser pour l'exercice 1



(a) Dessinez le BST obtenu après avoir inséré, dans cet ordre, les clés **35**, **75**, **45**.

(b) En repartant de l'arbre de départ ci-dessus (sans les insertions de la question a), dessinez l'arbre obtenu après **suppression de la clé 70**. Explicitez quel cas (feuille / 1 enfant / 2 enfants) s'applique et quelle est la clé qui remplace 70.

Cas de suppression : _____

Clé de remplacement : _____

(c) Donnez le résultat du parcours *inorder* de l'arbre de départ (avant toute modification) :

Que constatez-vous ?

8 Exercice 2 — Hauteur et complexité

On insère successivement, dans un BST initialement vide, les $n = 15$ clés suivantes :

Séquence A : 8, 4, 12, 2, 6, 10, 14, 1, 3, 5, 7, 9, 11, 13, 15

Séquence B : 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15

(a) Sans dessiner l'arbre en entier, déterminez la hauteur obtenue dans chaque cas :

Hauteur avec séquence A : _____ Hauteur avec séquence B : _____

(b) Pour chaque séquence, donnez le nombre *exact* de comparaisons effectuées par `get(15)` (nombre de nœuds visités avant de trouver la clé 15) :

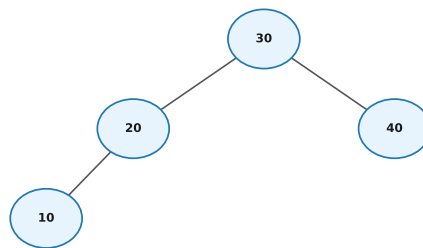
Comparaisons séquence A : _____ Comparaisons séquence B : _____

(c) Complétez : pour un BST à n nœuds, la complexité de `get` est en $\Theta(\text{_____})$ où ce symbole désigne _____. Dans le pire cas, cette quantité vaut $\Theta(\text{_____})$. Dans un BST équilibré, elle vaut $\Theta(\text{_____})$.

9 Exercice 3 — Rotations AVL

On considère l'arbre AVL suivant (les facteurs d'équilibre sont dans les limites autorisées) :

Arbre AVL avant insertion de 5 — à utiliser pour l'exercice 3



(a) On insère la clé 5. Dessinez l'arbre obtenu *immédiatement après l'insertion*, avant toute rotation.

(b) Identifiez le nœud déséquilibré et calculez son facteur d'équilibre.

Nœud déséquilibré : _____ Facteur d'équilibre : _____

(c) Quel cas (LL / RR / LR / RL) s'applique, et quelle rotation (ou combinaison de rotations) doit-on effectuer ?

Cas : _____ Rotation(s) : _____

(d) Dessinez l'arbre final après rééquilibrage.

10 Exercice 4 — Arithmétique des indices dans un tas

Un min-heap est stocké dans un tableau indexé à partir de 0. La taille courante est $n = 12$.

(a) Pour le nœud d'indice 4, donnez les indices de son parent, son enfant gauche et son enfant droit :

parent = _____ gauche = _____ droite = _____

(b) Les feuilles de ce heap occupent les indices :

(c) Le nœud d'indice 8 est-il une feuille ? Justifiez :

(d) Quelle est la hauteur de ce heap ? _____

Aide : pour un heap à n éléments, la hauteur est $\lfloor \log_2 n \rfloor$.

11 Exercice 5 — Percolation dans un min-heap

On part du min-heap stocké dans le tableau : [3, 5, 8, 7, 9, 10, 12].

(a) Dessinez l'arbre binaire correspondant (en indiquant la valeur et l'indice de chaque nœud).

(b) On appelle `insert(2)`. Donnez les états successifs du tableau pendant la percolation vers le haut, jusqu'à stabilisation :

Étape	État du tableau
Ajout en queue	
Après échange 1	
Après échange 2	
Après échange 3	

(c) Sur l'arbre final obtenu, on appelle maintenant `extractMin()`. Donnez :

La valeur retournée : _____

L'état du tableau après extraction et percolation vers le bas :

12 Exercice 6 — Pièges conceptuels

Répondez par **vrai** ou **faux** et justifiez en une ligne.

(a) Dans un min-heap, la deuxième plus petite valeur se trouve forcément à l'indice 1 ou à l'indice 2.

Réponse : _____

Justification : _____

(b) Un parcours inorder sur un min-heap donne les clés triées.

Réponse : _____

Justification : _____

(c) Toute insertion dans un AVL provoque au plus deux rotations.

Réponse : _____

Justification : _____

(d) Un BST dans lequel tous les nœuds ont 0 ou 2 enfants (arbre strict) est forcément équilibré.

Réponse : _____

Justification : _____

13 Pour aller plus loin (optionnel)

Si vous avez fini et que vous voulez creuser :

- Ajoutez la méthode `delete(K cle)` à `SimpleBST` en traitant les 3 cas (feuille, 1 enfant, 2 enfants avec successeur inorder).
- Rendez `SimpleBST` itérable pour pouvoir faire `for (K cle : bst)` et obtenir les clés triées (utilise la propriété inorder).
- Modifiez le programme d'observation pour mesurer aussi le *temps* de `get` sur une clé profonde (ex : la plus grande), et confirmez empiriquement la dégradation en $\mathcal{O}(n)$ dans le cas trié.
- Implémentez une version *heapify* $\mathcal{O}(n)$ qui construit un tas à partir d'un tableau existant (cf. CLRS §6.3 Build-Max-Heap).

14 Aide-mémoire

Aide-mémoire S04

Concept	Résumé
Propriété BST	gauche < nœud < droite (à tous les niveaux)
Inorder d'un BST	donne les clés triées
Hauteur BST	$h \in [\lfloor \log_2 n \rfloor, n - 1]$ selon l'ordre d'insertion
Complexité BST	put/get/contains en $\mathcal{O}(h)$
Invariant AVL	$ \text{BF}(x) \leq 1$ pour tout nœud x
Cas AVL	LL, RR (1 rotation); LR, RL (2 rotations)
Propriété min-heap	parent $\leq$ enfants
Indices (base 0)	parent de $i = \lfloor (i - 1)/2 \rfloor$, enfants en $2i + 1$ et $2i + 2$
peekMin	$\mathcal{O}(1)$ — toujours à l'indice 0
insert, extractMin	$\mathcal{O}(\log n)$ via percolation

Commandes utiles

Commande	Usage
<code>gradle -no-daemon build</code>	Compiler le projet
<code>gradle -no-daemon test</code>	Exécuter les tests JUnit
<code>gradle -no-daemon run</code>	Exécuter la classe Main
<code>git add . && git commit -m "msg"</code>	Committer les changements

Rapport individuel

Rapport individuel — obligatoire

Chaque étudiant rédige un rapport de **1 à 2 pages** :

1. **Ce que j'ai fait** : classes implémentées, tests écrits, ce que Main affiche.
2. **Les observations clés** : qu'est-ce que le programme d'observation m'a montré que je n'avais pas bien compris du CM? (en particulier, comment la hauteur varie selon l'ordre d'insertion)
3. **Les difficultés** : erreurs de compilation, bugs, blocages sur les exercices théoriques.
4. **Ce que j'ai appris** : le concept le plus important retenu de cette séance.

Format : PDF. **Nom** : rapport-lab-S04-PRENOM-NOM.pdf